PRD — Kumi TTPN Admin Platform

Versión: 2.0.0
Fecha: 2026-04-11
Empresa: Transportación Turística y Privada del Norte (TTPN) / Grupo Tulpe SA de CV
Audiencia: Equipos de desarrollo que deben replicar o extender el sistema desde cero

---

Tabla de Contenidos

1. Visión General del Producto
2. Stack Tecnológico
3. Arquitectura
4. Modelos de Datos Centrales
5. Autenticación y Permisos
6. Módulo: Dashboard
7. Módulo: Empleados
8. Módulo: Clientes
9. Módulo: Vehículos y Flotilla
10. Módulo: Servicios TTPN
11. Módulo: Cuadre y Captura
12. Módulo: Combustible
13. Módulo: Nómina
14. Módulo: Finanzas (Facturación)
15. Módulo: Proveedores
16. Módulo: Alertas
17. Módulo: Solicitudes de Chofer
18. Módulo: Citas a Empleados
19. Módulo: Configuración (Settings)
20. App Móvil — Contexto y Migración
21. Integraciones y APIs Externas
22. Infraestructura y Despliegue
23. Lógica de Negocio Crítica
24. Reglas y Restricciones Globales

---

1. Visión General del Producto

Kumi TTPN Admin es la plataforma de gestión operativa de TTPN, empresa de transportación turística y privada con sede en Chihuahua, México. El sistema centraliza:

- Gestión de choferes y empleados
- Asignación de vehículos
- Captura y cuadre de servicios de transporte (travel counts vs bookings)
- Nómina de choferes
- Facturación a clientes

- Control de combustible
- Alertas operativas

El sistema opera con **dos unidades de negocio**:

ID	Nombre
1	Transportación Turística y Privada del Norte
2	Grupo Tulpe SA de CV

Cada registro en casi toda la BD pertenece a una `business_unit_id`. Los usuarios solo ven datos de su propia unidad de negocio, salvo el rol `sistemas` (sadmin) que puede ver todo.

---

2. Stack Tecnológico

Backend

Componente	Tecnología
Framework	Ruby on Rails 7.1 (API mode)
Base de datos	PostgreSQL (Supabase Pro en producción)
Autenticación	Devise + JWT (gem `devise-jwt`)
Jobs asíncronos	Sidekiq + Redis
Caching	Redis
Documentación API	Rswag (Swagger UI en `/api-docs`)
Gestión de memoria	`puma_worker_killer`
Autorización	Custom (Privilege model + role_privileges)
Almacenamiento archivos	ActiveStorage (Supabase Storage en prod)

Frontend

Componente	Tecnología
Framework	Quasar Framework v2 (Vue 3 + Vite)
Lenguaje	JavaScript (Composition API)
UI Library	Quasar Components
HTTP	Axios (proxy a `/api`)
Hosting prod	Netlify
Hosting dev	Docker (puerto 9000)

Infraestructura Producción

Servicio	Uso
Railway	Rails API + Redis + Sidekiq + N8N
Supabase Pro ($25/mes)	PostgreSQL + Storage
Netlify	Frontend (Quasar)
GitHub	Repositorio FE (castean/kumi-frontend)
GitLab	Repositorio BE (ttpngas)

---

3. Arquitectura

```
┌──────────────────────────────────────────────────────────────┐
│              MONOLITO MODULAR (Rails 7.1 API)                 │
│  Un solo proceso · Una sola BD · Transacciones ACID           │
│                                                               │
│  Empleados · Vehículos · Viajes · Nómina                     │
│  Clientes  · Bookings  · Gas    · Alertas                    │
└──────────────────────────────────────────────────────────────┘
                            │ REST JSON API
          ┌─────────────────┼─────────────────┐
          ▼                 ▼                 ▼
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│  Kumi Admin  │   │  App Móvil   │   │   N8N        │
│  (Quasar)    │   │  (Android/   │   │ Automatiza-  │
│  Netlify     │   │   PWA futura)│   │ ciones       │
└──────────────┘   └──────────────┘   └──────────────┘
  Usuarios admin    Choferes           M2M vía api_keys
```

Namespacing de Rutas

```
/api/v1/auth/*              → Autenticación usuarios admin
/api/v1/*                   → Todos los módulos (JWT de User)
/api/v1/mobile/auth/*       → Autenticación choferes (futuro)
/api/v1/mobile/*            → Endpoints app móvil (JWT de Employee)
```

Estructura de Directorios (BE)

```
ttpngas/
├── app/
│   ├── controllers/api/v1/     → Todos los controllers REST
│   ├── models/                 → ActiveRecord models
│   ├── jobs/                   → Sidekiq jobs
│   ├── services/               → Service objects
│   └── helpers/                → Módulos de lógica reutilizable
├── config/routes/              → Rutas divididas por dominio
```

```
│   ├── auth.rb
│   ├── dashboard.rb
│   ├── vehicles.rb
│   ├── employees.rb
│   ├── clients.rb
│   ├── bookings.rb
│   ├── payroll.rb
│   ├── fuel.rb
│   ├── administration.rb
│   └── alerts.rb
├── db/
│   ├── migrate/                → Migraciones (include funciones PG)
│   └── seeds/                  → Seeds por dominio
└── documentacion/              → Docs técnicas del proyecto
```

Estructura de Directorios (FE)

```
ttpn-frontend/
├── src/
│   ├── pages/                  → Páginas por módulo
│   ├── components/             → Componentes reutilizables
│   ├── composables/            → Lógica reutilizable (useX)
│   ├── services/               → Clientes HTTP (axios)
│   ├── boot/                   → Inicialización (axios, auth)
│   └── router/                 → Vue Router
```

---

4. Modelos de Datos Centrales

Entidades principales y sus relaciones

```
BusinessUnit
  ├── has_many :users
  ├── has_many :employees
  ├── has_many :vehicles (indirecta)
  └── has_many :kumi_settings

User
  ├── belongs_to :business_unit
  ├── belongs_to :role
  └── string[] :allowed_apps (ej. ["admin_web"])

Employee
  ├── belongs_to :business_unit
  ├── belongs_to :labor (puesto)
  ├── belongs_to :concessionaire
  ├── has_many :employee_movements
  ├── has_many :vehicle_asignations
  └── has_many :travel_counts
```

```
Vehicle
  ├── belongs_to :vehicle_type
  ├── has_many :vehicle_asignations
  ├── has_many :gas_charges
  └── has_many :vehicle_checks

VehicleAsignation
  ├── belongs_to :vehicle
  ├── belongs_to :employee
  └── fecha_hasta: nil → asignación activa

Client
  ├── has_many :client_branch_offices
  ├── has_many :client_contacts
  └── has_many :ttpn_services

TtpnService (servicio contratado por cliente)
  ├── belongs_to :client
  ├── belongs_to :ttpn_foreign_destiny
  └── has_many :ttpn_bookings

TtpnBooking (programación de servicio)
  ├── belongs_to :client
  ├── belongs_to :ttpn_service
  ├── belongs_to :employee
  ├── belongs_to :vehicle
  └── belongs_to :travel_count (cuadre)

TravelCount (captura del chofer)
  ├── belongs_to :employee
  ├── belongs_to :vehicle
  ├── belongs_to :client_branch_office
  ├── belongs_to :ttpn_booking (cuadre)
  └── belongs_to :payroll

Payroll
  └── has_many :travel_counts

GasCharge
  ├── belongs_to :vehicle
  └── belongs_to :gas_station

Discrepancy
  └── polymorphic :record (TravelCount o GasCharge)
```

---

5. Autenticación y Permisos

Autenticación de Usuarios (Admin Web)

- Devise + JWT. Al login, se genera un JWT con `{ user_id, jti, exp: 24h }`.

- El `jti` se guarda en `users.jti`. Al logout, se regenera el jti (revocación inmediata).
- Cada request debe incluir `Authorization: Bearer <token>`.
- El `BaseController` valida el JWT y lo cruza con `users.jti`.

Endpoint:

```
POST /api/v1/auth/login
Body: { email, password }
Response: { access_token, token_type, expires_in, user, privileges }
```

Sistema de Roles y Privilegios

Hay dos capas de autorización:

Capa 1 — Roles:

ID	Rol	Descripción
1	sistemas	SuperAdmin — acceso total, sin filtro de BU
2	admin	Administrador de unidad de negocio
3	rh	Recursos Humanos
4	coordinador	Coordinador de servicios
5	capturista	Solo captura de servicios
6	mantenimiento	Solo módulo de vehículos
19	as_direccion	Asistente de Dirección
21	monitoreo	Solo lectura operativa
22	rh_tulpe	RH unidad Tulpe
23	coordinador_tulpe	Coordinador unidad Tulpe
24	aux_coordinacion	Auxiliar de coordinación

Capa 2 — Privilegios por módulo:

El modelo `Privilege` define qué acciones están disponibles en cada módulo:

```
Privilege {
  module_key:      string   # identificador único (ej. "travel_counts")
  module_name:     string   # nombre legible
  module_group:    string   # grupo (ej. "Cuadre y Captura")
  route_path:      string   # path del FE
  requires_access: boolean  # ver el módulo
  requires_create: boolean
  requires_edit:   boolean
  requires_delete: boolean
```

```
    requires_clone:  boolean
    requires_import: boolean
    requires_export: boolean
  }
```

Los privilegios se asignan a roles mediante `RolePrivilege`. Al hacer login, el backend retorna el hash `privileges` con los permisos activos del usuario.

Filtro de Business Unit:

- `sadmin? == true` → puede ver todas las BUs o filtrar por `?business_unit_id=`
- Demás roles → siempre ven solo su propia `business_unit_id`
- Implementado en `BaseController#current_business_unit_id`

Multi-app (registered_apps)

`users.allowed_apps` es un array de strings que define a qué aplicaciones puede acceder el usuario:

```
["admin_web"]           # Solo el panel admin
["admin_web", "movil"]  # Panel + app móvil (futuro)
```

---

6. Módulo: Dashboard

Ruta FE: `/`
Controller: `Api::V1::DashboardController`
Privilege: `dashboard`

Función

Pantalla principal con KPIs en tiempo real del día actual.

KPIs mostrados

KPI	Fuente
Viajes capturados hoy	`TravelCount.where(fecha: Date.today).count`
Viajes cuadrados hoy	Función PG `enc_travel()`
Bookings programados hoy	`TtpnBooking.where(fecha: Date.today).count`
Bookings cuadrados	Función PG `enc_booking()`
Vehículos activos	`Vehicle.where(status: true).count`
Choferes activos	`Employee.where(status: true).count`
Alertas sin resolver	`Alert.where(status: 'pending').count`

Dashboard de Cuadre (TtpnDashboard)

Controller separado `Api::V1::TtpnDashboardController`. Muestra el estado del cuadre por fecha con funciones PG:

- `cont_viajes(fecha)` → total travel_counts
- `cont_capt(fecha)` → bookings capturados
- `cost_viajes(fecha)` → costo total de viajes
- `cobro_fact(fecha)` → total a facturar

---

7. Módulo: Empleados

Ruta FE: `/employees`
Controllers: `EmployeesController`, `EmployeeVacationsController`, `AguinaldosController`, `EmployeeAppointmentsController`, `EmployeeDeductionsController`, `EmployeesIncidencesController`

7.1 Directorio de Empleados

Tabla principal: `employees`

Campo	Tipo	Descripción
`clv`	string	Clave única del empleado (ej. "00009")
`nombre`	string	Nombre(s)
`apaterno`	string	Apellido paterno
`amaterno`	string	Apellido materno
`sexo`	string	masculino / femenino
`fecha_nacimiento`	date	Usado para autenticación móvil
`labor_id`	integer	FK → Labor (puesto)
`status`	boolean	true = activo
`imei`	string	IMEI/AndroidID del dispositivo (legacy)
`app_version`	string	Última versión de app usada
`business_unit_id`	integer	FK → BusinessUnit
`concessionaire_id`	integer	FK → Concessionaire
`jti`	string	(pendiente migrar) Para JWT móvil

Status activo: `status: true` + último `EmployeeMovement` de tipo Alta (1) o Reingreso (3).

Puestos (Labor): Chofer, Coordinador, Capturista, Recursos Humanos, Sistemas, etc.

7.2 Movimientos de Empleado

Tabla: `employee_movements`

Tipo	ID	Efecto
Alta	1	Activa al empleado
Baja	2	Desactiva (status = false)
Reingreso	3	Reactiva al empleado
Cambio de puesto	4	Solo registro histórico
Incremento salarial	5	Solo registro histórico
Baja - Lista Negra	6	Desactiva, no puede reingresar

El método `elegible_para_operar?` en `Employee` verifica que el último movimiento sea de tipo 1 o 3 (`EmployeeMovementType.altas_y_reingresos_ids`).

7.3 Vacaciones

Tabla: `employee_vacations`

- Cálculo de días correspondientes según años trabajados (tabla configurable en `KumiSetting`)
- Método `Employee#dias_vacaciones_pendientes` cruza `KumiSetting.vacation_days_for_year` con días ya tomados

7.4 Aguinaldo

Controller `AguinaldosController` — calcula el aguinaldo según días trabajados en el año y el SDI (salario diario integrado) de `employee_salaries`.

7.5 Deducciones

Tabla: `employee_deductions`
Descuentos aplicados a nómina (préstamos, uniformes, herramientas, etc.)

7.6 Incidencias

Tabla: `employees_incidences`
Registro de faltas, retardos, sanciones. Se cruza con nómina.

7.7 Citas a Empleados

Tablas: `employee_appointments`, `employee_appointment_attendees`, `employee_appointment_logs`
Gestión de citas médicas, psicológicas, administrativas para empleados. Permite clonar citas recurrentes.

---

8. Módulo: Clientes

Ruta FE: `/clients`
Controllers: `ClientsController`, `ClientBranchOfficesController`, `ClientUsersController`

Modelo Client

Campo	Descripción
`clv`	Clave única del cliente
`razon_social`	Nombre legal
`rfc`	RFC fiscal
`calle`, `numero`, `colonia`, `ciudad`, `estado`, `codigo_postal`	Dirección
`status`	true = activo
`business_unit_id`	FK → BusinessUnit

Modelo ClientBranchOffice (Planta del cliente)

Importante: Las plantas tienen coordenadas GPS (`lat`, `lng`) usadas por la app móvil para auto-sugerir la planta más cercana al chofer (radio 300 metros, algoritmo Haversine).

Campo	Descripción
`client_id`	FK → Client
`nombre`	Nombre de la planta
`lat`	Latitud
`lng`	Longitud
`ttpn_foreign_destiny_id`	Destino por defecto para viajes desde esta planta

Modelo ClientUser

Usuarios del cliente (para portal de clientes futuro). FK a `Client` y tienen acceso restringido.

Submodelos relacionados

- `ClientContact` — contactos del cliente
- `ClientBranchOfficeContact` — contactos por planta
- `ClientBranchOfficeTtpnContact` — contactos TTPN asignados a cada planta
- `ClientEmployee` — empleados del cliente (pasajeros frecuentes)
- `ClientTtpnService` — servicios TTPN contratados por el cliente

---

9. Módulo: Vehículos y Flotilla

Ruta FE: `/vehicles`, `/vehicle-asignations`, `/vehicle-checks`, `/driver-requests`
Controllers: `VehiclesController`, `VehicleAsignationsController`, `VehicleChecksController`, `DriverRequestsController`

Modelo Vehicle

Campo	Descripción

Campo	Descripción
`clv`	Clave única (ej. "T035")
`modelo`	Modelo del vehículo
`marca`	Marca
`serie`	Número de serie
`placa`	Placas
`annio`	Año
`vehicle_type_id`	FK → VehicleType (Van, Auto, Camioneta, etc.)
`password`	Contraseña para app móvil (formato: `ttpnXX`)
`status`	true = activo en operación
`gps_uniq`	ID de dispositivo GPS

Formato de password: `"ttpn"` + número después del primer cero en `clv`.
Ejemplo: `clv = "T035"` → `password = "ttpn35"`, `clv = "T001"` → `password = "ttpn1"`.

Modelo VehicleAsignation (Asignación Chofer-Vehículo)

Campo	Descripción
`vehicle_id`	FK → Vehicle
`employee_id`	FK → Employee
`fecha_efectiva`	Inicio de la asignación
`fecha_hasta`	Fin (`nil` = asignación activa en curso)

Regla de negocio: Solo puede haber una asignación activa por vehículo a la vez. Al asignar un nuevo chofer, se cierra la asignación anterior (`fecha_hasta = NOW()`).

Modelo VehicleCheck (Revisión del Vehículo)

Inspecciones del estado físico del vehículo. Tiene tres etapas:

Etapa	Campo	Quién la realiza
Origen	`puntos_originales`, `fecha_origen`	Coordinador al salir
Revisión	`puntos_revisados`, `fecha_revision`	Chofer al regresar
Auditoría	`puntos_auditados`, `fecha_auditoria`	Coordinador/Admin

Los `puntos_*` son JSONs con el estado de cada punto de revisión (configurable en `ReviewPoint`).

Tabla: `review_points` — catálogo de puntos de revisión (ej. Llantas, Limpiaparabrisas, Espejo izquierdo, etc.)

Modelo DriverRequest (Solicitudes de Mantenimiento)

Los choferes reportan fallas o solicitudes de mantenimiento desde la app. Campos: `vehicle_id`, `employee_id`, `descripcion`, `status`.

Catálogos de Vehículos

Modelo	Descripción
`VehicleType`	Tipo (Van, Auto, Camioneta, etc.)
`VehicleDocument`	Documentos del vehículo (póliza, factura, etc.)
`VehicleDocumentType`	Catálogo de tipos de documento
`VehicleTypePrices`	Precio base por tipo de vehículo
`ScheduledMaintenance`	Mantenimientos programados

---

10. Módulo: Servicios TTPN

Ruta FE: `/ttpn_services`, `/services/*`
Controllers: `TtpnServicesController`, `TtpnServiceTypesController`, `TtpnForeignDestiniesController`, `TtpnServiceDriverIncreasesController`

Estructura

```
TtpnForeignDestiny (Destino)
  ↑
TtpnService (Servicio contratado)
  ├── belongs_to :client
  ├── belongs_to :ttpn_foreign_destiny
  └── has_many :ttpn_bookings

TtpnServiceType (Tipo de servicio)
  ID 1: Entrada
  ID 2: Salida
```

Modelo TtpnService

Representa un servicio específico contratado por un cliente. Por ejemplo: "Servicio RTA Planta Norte — Entrada".

Campo	Descripción
`clv`	Clave única del servicio
`descripcion`	Nombre del servicio
`ttpn_foreign_destiny_id`	Destino del servicio

Campo	Descripción
`status`	true = activo

Modelo TtpnForeignDestiny (Destino)

Destinos operativos de TTPN (plantas, aeropuertos, hoteles, etc.).

TtpnServiceDriverIncrease

Incremento de costo por asignación de chofer específico a un servicio. Permite configurar tarifas personalizadas por combinación servicio-chofer.

TtpnServicePrice

Precios configurables por servicio.

---

11. Módulo: Cuadre y Captura

Ruta FE: `/ttpn_bookings/*`, `/travel_counts`
Controllers: `TtpnBookingsController`, `TravelCountsController`, `DiscrepanciesController`

Este es el módulo más crítico del sistema. El "cuadre" es el proceso de hacer match entre lo que se programó (booking) y lo que el chofer capturó (travel count).

11.1 TtpnBooking (Programación de Servicio)

Un coordinador registra los servicios programados para el día. Al crearse o modificarse, el sistema intenta automáticamente hacer match con un `TravelCount` existente.

Columnas clave:

Campo	Descripción
`client_id`	Cliente del servicio
`fecha`	Fecha del servicio
`hora`	Hora del servicio
`ttpn_service_type_id`	Tipo (Entrada=1, Salida=2)
`ttpn_service_id`	Servicio específico
`vehicle_id`	Vehículo asignado
`employee_id`	Chofer asignado
`clv_servicio`	Llave de cuadre (sin service_id)
`clv_servicio_completa`	Llave de cuadre (con service_id)
`viaje_encontrado`	boolean — si ya hizo match con TravelCount

Campo	Descripción
`travel_count_id`	FK → TravelCount cuadrado
`passenger_qty`	Cantidad de pasajeros
`empalme`	boolean — servicio empalme
`creation_method`	manual / cloned / imported

Formato clv_servicio:

```
{client_id}-{fecha}-{hora HH:MM:SS}-{ttpn_service_type_id}-
{foreign_destiny_id}-{vehicle_id}
```

Se usa para el cuadre automático. Los segundos diferencian viajes dobles en la misma hora.

Callbacks del modelo:

- `before_validation :extra_campos` → calcula `clv_servicio`, asigna `employee_id` desde asignación vigente
- `before_create :statuses` → intenta cuadre inicial con `busca_en_travel()`
- `after_create :create_actualiza_tc` → si cuadró, actualiza el TravelCount como `viaje_encontrado: true`
- `before_update :update_borra_tc` → al editar, deshace el cuadre anterior y reintenta
- `before_destroy :borra_tc_destroy` → al eliminar, libera el TravelCount

11.2 TravelCount (Captura del Chofer)

El chofer registra cada viaje realizado desde la app móvil. Es el registro base para nómina.

Columnas clave:

Campo	Descripción
`employee_id`	Chofer que realizó el viaje
`vehicle_id`	Vehículo usado
`client_branch_office_id`	Planta del cliente
`ttpn_service_type_id`	Tipo (Entrada/Salida)
`ttpn_foreign_destiny_id`	Destino del viaje
`fecha`	Fecha del viaje
`hora`	Hora del viaje
`costo`	Costo calculado del viaje
`viaje_encontrado`	boolean — si cuadró con un booking

Campo	Descripción
`ttpn_booking_id`	FK → TtpnBooking cuadrado
`payroll_id`	FK → Payroll (nómina donde se incluye)
`aforo`	Cantidad de pasajeros
`requiere_autorizacion`	boolean — si el admin debe aprobar
`status_autorizacion`	pending / approved / rejected
`errorcode`	Código de error del cuadre

11.3 Trigger `sp_tctb_insert` (PostgreSQL)

El trigger más crítico del sistema. Se ejecuta al insertar un `TravelCount` y hace automáticamente:

1. `buscar_booking()` → busca si ya existe un booking que haga match
2. `buscar_booking_id()` → obtiene el ID del booking
3. `buscar_nomina()` → asigna el `payroll_id` de la nómina vigente

```
-- Simplificado:
NEW.viaje_encontrado := buscar_booking(vehicle_id, employee_id, ...)
NEW.ttpn_booking_id  := buscar_booking_id(vehicle_id, employee_id, ...)
NEW.payroll_id       := buscar_nomina()
```

11.4 Trigger `sp_tctb_update`

Mismo proceso pero al actualizar un TravelCount.

11.5 Trigger `tc_insert` / `tc_update`

Triggers en `travel_counts` que actualizan el `TtpnBooking` cuadrado (campo `travel_count_id` y `viaje_encontrado`).

11.6 Discrepancias

Tabla: `discrepancies`

Una discrepancia es un TravelCount o GasCharge que no pudo cuadrarse automáticamente. El sistema las crea automáticamente vía `resolver_discrepancia_pnc`.

Campo	Descripción
`record_type`	Polymorphic: "TravelCount" o "GasCharge"
`record_id`	ID del registro problemático
`kpi`	Tipo de discrepancia
`status`	pendiente / resuelta

Campo	Descripción
`descripcion`	Descripción de la discrepancia

Vista de cuadre en FE: Muestra bookings y travel counts lado a lado, con semáforo visual:

- Verde: cuadrado automáticamente
- Amarillo: cuadrado manualmente
- Rojo: sin cuadrar (discrepancia PNC)

---

12. Módulo: Combustible

Ruta FE: `/gas/*`
Controllers: `GasChargesController`, `GasolineChargesController`, `GasStationsController`, `FuelPerformanceController`

12.1 GasCharge (Carga de Gas — GNV)

El chofer registra cada carga de gas natural desde la app móvil.

Campo	Descripción
`vehicle_id`	Vehículo
`monto`	Monto pagado
`cantidad`	Cantidad en m³
`odometro`	Lectura del odómetro al cargar
`fecha`, `hora`	Cuándo se cargó
`lat`, `lng`	Coordenadas GPS donde se cargó
`gas_station_id`	Estación de gas (si se pudo identificar)
`ticket`	Número de ticket
`carga_encontrada`	boolean — si cuadró con gas_file
`gas_file_id`	FK → GasFile (archivo de la gasera)
`fecha_cuadre`	Cuándo se cuadró

12.2 GasFile (Archivo de la Gasera)

El administrador sube el archivo de facturación de la gasera. El sistema intenta cuadrar cada línea con un `GasCharge` existente.

Función PG: `buscar_gascharge_id()` — hace el match automático entre líneas del archivo y cargas registradas por choferes.

12.3 GasolineCharge (Carga de Gasolina)

Similar a `GasCharge` pero para vehículos a gasolina. Tabla separada.

12.4 GasStation (Estaciones de Gas)

Catálogo de estaciones con coordenadas GPS. Se usa para el mapa interactivo en la app móvil.

12.5 FuelPerformance (Rendimientos)

Cálculo de rendimiento de combustible por vehículo:

- km recorridos entre cargas (delta odómetro)
- m^3 consumidos
- km/m^3 (rendimiento)

---

13. Módulo: Nómina

Ruta FE: `/ttpn_bookings/payroll`
Controllers: `PayrollsController`, `PayrollReportsController`

Modelo Payroll

Una nómina es un período de tiempo durante el cual se acumulan los `TravelCount` de los choferes.

Campo	Descripción
`descripcion`	Nombre del período (ej. "Nómina Quincena 1 Abril 2026")
`fecha_inicio`	Inicio del período
`hora_inicio`	Hora de inicio
`fecha_hasta`	Fin del período
`hora_hasta`	Hora de fin
`fecha_fin_planificada`	Cuándo se planea cerrar
`processing_status`	pending / processing / completed / error
`dispercion`	Archivo de dispersión bancaria
`excel_generated`	boolean — si ya se generó el Excel
`progress`	0-100 progreso del cálculo

Flujo de Nómina

```
1. Admin crea Payroll (fecha_inicio + fecha_hasta)
2. Los TravelCount del período se asignan automáticamente
   vía función PG buscar_nomina() en el trigger sp_tctb_insert
3. Admin ejecuta el cálculo de la nómina (Sidekiq job)
4. El sistema calcula:
   - pago_chofer(employee_id, payroll_id) → pago base
```

```
    - incremento_por_nivel(employee_id) → bono por nivel
    - incremento_cliente(client_id) → bono por cliente
    - incremento_servicio(service_id) → bono por servicio
    - deducciones del período
  5. Se genera el Excel de dispersión
  6. Admin exporta y sube al banco
```

PayrollLog

Auditoría del proceso de generación de nómina. Cada paso del cálculo queda registrado.

Niveles de Choferes (DriverLevel)

Los choferes tienen niveles (Bronce, Plata, Oro, etc.) que definen el incremento en su pago:

- `employee_drivers_levels` → historial de niveles del chofer
- `drivers_levels` → catálogo de niveles con porcentajes de incremento
- `cts_increments` / `cts_increment_details` → detalles de incrementos por período

---

14. Módulo: Finanzas (Facturación)

Ruta FE: `/ttpn_bookings/invoicing`
Controller: `InvoicingsController`

Modelo Invoicing

Generación de reportes de facturación para los clientes.

Campo	Descripción
`fecha_inicio`, `fecha_hasta`	Período de facturación
`invoice_type_id`	Tipo de factura
`client_id`	Cliente a facturar
`periodicidad`	semanal / quincenal / mensual
`report_status`	pending / processing / completed / error
`report_filename`	Nombre del archivo generado

Proceso

El sistema agrupa los `TtpnBooking` cuadrados del período por cliente y calcula el monto total usando:

- `cobro_fact(fecha)` — función PG que suma costos de todos los bookings cuadrados
- `cuenta_cap_fact()` — cuenta cuántos bookings están listos para facturar

La factura se genera como Excel/PDF vía Sidekiq job.

InvoiceType

Catálogo de tipos de factura (por servicio, por cliente, por período, etc.).

---

15. Módulo: Proveedores

Ruta FE: `/suppliers`
Controller: `SuppliersController`

Directorio de proveedores para el área de mantenimiento y compras. Campos básicos: nombre, RFC, teléfono, correo, categoría.

---

16. Módulo: Alertas

Ruta FE: `/alerts`
Controllers: `AlertsController`, `AlertRulesController`, `AlertContactsController`

Arquitectura del Sistema de Alertas

```
AlertRule (define cuándo disparar)
  ↓ trigger_condition se cumple
Alert (se crea la alerta)
  ↓ proceso de entrega
AlertDelivery (canal de entrega: email, sms, push)
  ↓
AlertContact (quién la recibe)
```

Modelo AlertRule

Reglas configurables que definen cuándo se dispara una alerta.

Campo	Descripción
`trigger_type`	Tipo de evento (viaje sin cuadrar, discrepancia, etc.)
`conditions`	JSON con condiciones específicas
`severity`	info / warning / critical
`business_unit_id`	BU que aplica

Modelo Alert

Campo	Descripción
`alert_rule_id`	Regla que la disparó
`trigger_type`	Tipo de alerta
`origin`	Sistema que la originó
`source_type`	Polymorphic: TravelCount, Vehicle, Employee, etc.

Campo	Descripción
`source_id`	ID del registro que originó la alerta
`titulo`	Título de la alerta
`descripcion`	Detalle
`status`	pending / read / resolved
`triggered_at`	Cuándo se disparó
`resolved_at`	Cuándo se resolvió
`resolved_by`	User que la resolvió

Campana de Notificaciones (In-App)

La campana en el header del FE consulta:

```
GET /api/v1/alerts?status=pending&per_page=20
```

y muestra el conteo de alertas no leídas. Este mecanismo reemplaza las notificaciones push de FCM para la web.

AlertRead

Registro de qué usuario leyó qué alerta.

---

17. Módulo: Solicitudes de Chofer

Ruta FE: `/driver-requests`
Controller: `DriverRequestsController`

Los choferes reportan desde la app móvil fallas mecánicas, solicitudes de mantenimiento o incidentes del vehículo.

Campo	Descripción
`vehicle_id`	Vehículo afectado
`employee_id`	Chofer que reporta
`descripcion`	Descripción del problema
`status`	pendiente / en_proceso / resuelto
`business_unit_id`	BU

El administrador ve todas las solicitudes y las asigna al área de mantenimiento.

---

18. Módulo: Citas a Empleados

Ruta FE: `/employees/appointments`
Controller: `EmployeeAppointmentsController`

Sistema de agenda para programar citas con empleados (medicina del trabajo, psicología, IMSS, capacitación, etc.).

Modelo	Descripción
`EmployeeAppointment`	La cita (fecha, hora, tipo, lugar)
`EmployeeAppointmentAttendee`	Qué empleados asistirán
`EmployeeAppointmentLog`	Auditoría de cambios de estado

Funcionalidad de clonar: Permite duplicar una cita con todos sus asistentes para reprogramarla.

---

19. Módulo: Configuración (Settings)

Ruta FE: `/settings`
Controllers: Múltiples controllers de catálogos

Settings está dividido en 6 secciones:

19.1 Catálogos de Empleados

Catálogo	Modelo	Descripción
Puestos	`Labor`	Catálogo de puestos de trabajo
Niveles de Choferes	`DriversLevel`	Niveles (Bronce, Plata, Oro, etc.) con % incremento
Tipos de Documentos Empleado	`EmployeeDocumentType`	Catálogo de documentos requeridos
Tipos de Incidencias	`Incidence`	Catálogo de tipos de incidencia
Tipos de Movimientos	`EmployeeMovementType`	Alta, Baja, Reingreso, etc.

19.2 Catálogos Vehiculares

Catálogo	Modelo	Descripción
Tipos de Vehículos	`VehicleType`	Van, Auto, Camioneta, etc.
Precios por Tipo	`VehicleTypePrices`	Tarifa base por tipo de vehículo
Tipos de Documentos Vehículo	`VehicleDocumentType`	Catálogo de documentos requeridos
Puntos de Revisión	`ReviewPoint`	Checklist de inspección de vehículos

19.3 Usuarios y Permisos

Funcionalidad	Descripción
Usuarios	CRUD de `User` — email, nombre, rol, BU, apps permitidas
Roles	CRUD de `Role`
Privilegios por Rol	Asignación de `Privilege` a `Role` mediante `RolePrivilege`
Catálogo de Módulos	Lista de todos los `Privilege` disponibles

Regla: Un usuario solo puede gestionar usuarios de su propia BU (excepto sadmin).

19.4 Organización

Funcionalidad	Descripción
Unidades de Negocio	`BusinessUnit` — las dos BUs del sistema
Concesionarios	`Concessionaire` — propietarios de vehículos
Versiones de la App	`Version` — control de versiones de app móvil y web

Modelo Version — reglas:

1. Máximo 2 versiones activas por dispositivo
2. Solo una versión permanente (sin `fecha_fin`) activa por dispositivo
3. La versión permanente debe tener `fecha_inicio` mayor que la temporal
4. Si una versión tiene `fecha_fin <= hoy`, debe existir una permanente de reemplazo
5. Si la `fecha_fin` es futura, se puede establecer sin reemplazo (se creará después)

19.5 Integraciones

Funcionalidad	Modelo	Descripción
API Keys	`ApiKey`	Llaves para acceso M2M (N8N, integraciones)
API Users	`ApiUser`	Clientes API (N8N, etc.)

`ApiKey` tiene campo `permissions` (JSON o array) con los módulos/acciones permitidas.

19.6 Configuración General

Modelo KumiSetting — configuración por clave/valor por BU:

category	key	Descripción
nomina	vacation_days_year_X	Días de vacaciones por año trabajado
nomina	aguinaldo_days	Días de aguinaldo
general	timezone	Zona horaria de la BU
alertas	umbral_discrepancias	% máximo de discrepancias antes de alerta

---

20. App Móvil — Contexto y Migración

Documentado en detalle en:

- PLAN_MIGRACION_MOVIL.md — Plan de migración PHP → Rails
- AUTH_MOVIL_CHOFERES.md — Autenticación JWT para choferes
- ANALISIS_PWA_VS_APP_NATIVA.md — PWA vs app nativa

Estado Actual

- App Android nativa (Java) conectada a backend PHP en Heroku
- En proceso de migración a Rails API + Quasar PWA/Capacitor

Flujo de Login del Chofer

```
1. Chofer ingresa clave del vehículo (ej. "T035")
2. Sistema valida: password = "ttpn" + número de la clave
3. Si hay asignación activa, se cierra (fecha_hasta = NOW())
4. Chofer ingresa: Nombre, Apellido Paterno, Fecha de Nacimiento
5. Sistema busca Employee que coincida y esté activo (Alta/Reingreso)
6. Se crea nueva VehicleAsignation
7. Sistema emite JWT de chofer: { employee_id, vehicle_id, jti, exp: 7d }
```

Funcionalidades de la App Móvil

Módulo	Estado
Login con clave de vehículo	En migración
Captura de TravelCount	En migración
Auto-sugerencia de planta por GPS (Haversine, 300m)	En migración
Consulta de Bookings del día	En migración
Carga de GasCharge con GPS	En migración
Fotos en VehicleCheck (`<input capture="environment">`)	En migración
Solicitudes de mantenimiento (DriverRequest)	En migración
Mapa de estaciones de gas	En migración
Notificaciones via campana de alertas	Diseñado

---

21. Integraciones y APIs Externas

N8N (Automatizaciones)

- Servicio independiente en Railway
- Se conecta al Rails API usando `api_keys` (autenticación M2M)
- No usa el sistema de JWT de usuarios — usa su propio `ApiKey`
- Casos de uso: envío de reportes automáticos, sincronización con sistemas externos, notificaciones

Autenticación M2M (api_keys)

```
# ApiKey valida el header:
# X-Api-Key: <key>
# o
# Authorization: ApiKey <key>

ApiKey {
  name:          string   # Nombre de la integración
  key:           string   # Token hasheado
  permissions:   JSON     # Módulos y acciones permitidas
  active:        boolean
  expires_at:    datetime # nil = nunca expira
  api_user_id:   FK       # Sistema que la usa
  requests_count: integer # Contador de uso
  last_used_at:  datetime
}
```

ActiveStorage + Supabase Storage

Archivos adjuntos (avatares de empleados, fotos de vehicle_checks, documentos):

- En desarrollo: almacenamiento local
- En producción: Supabase Storage vía ActiveStorage adapter

Swagger/Rswag

Documentación interactiva de la API disponible en `/api-docs`. Cualquier desarrollador puede explorar y probar endpoints desde el navegador.

---

22. Infraestructura y Despliegue

Ver documento completo: INFRAESTRUCTURA_RAILWAY_NETLIFY_SUPABASE.md

Entornos

Entorno	BE	FE	BD
Desarrollo	Docker (localhost:3000)	Docker (localhost:9000)	PostgreSQL local
Producción	Railway	Netlify	Supabase Pro

Docker (Desarrollo)

```
# docker-compose.yml — servicios:
api:      Rails (puerto 3000)
frontend: Quasar (puerto 9000→9200)
redis:    Redis 7 (puerto 6379)
sidekiq:  Worker Sidekiq
```

Comandos clave:

```
# Iniciar todo
docker compose up

# Migrar BD
docker compose exec api bundle exec rails db:migrate

# Seeds
docker compose exec api bundle exec rails db:seed

# Consola Rails
docker compose exec api bundle exec rails console

# Ver logs
docker compose logs -f api
```

Variables de Entorno Requeridas (BE)

```
DATABASE_URL=postgresql://...
SECRET_KEY_BASE=...
REDIS_URL=redis://...
RAILS_ENV=production
```

Gestión de Memoria en Producción

`puma_worker_killer` configurado para reiniciar workers cuando superan un umbral de RAM:

```
PumaWorkerKiller.config { |c| c.ram = 512; c.percent_usage = 0.98 }
PumaWorkerKiller.start
```

Railway Health Check en `/up` con Restart Policy automático.

Costos Actuales vs Propuesta

	Heroku (actual)	**Railway + Supabase (propuesto)**
Servidor Rails	$252.88/mes (Performance-M)	~$20-30/mes (compute graduado)
BD PostgreSQL	Supabase Pro $25/mes	Supabase Pro $25/mes
Redis	Incluido	~$5/mes
Total	~$371/mes	~$55-70/mes

---

23. Lógica de Negocio Crítica

Cuadre Automático (el corazón del sistema)

El cuadre es el match entre lo que el coordinador programó (TtpnBooking) y lo que el chofer capturó (TravelCount).

Llave de cuadre:

```
{client_id}-{fecha}-{hora HH:MM:SS}-{ttpn_service_type_id}-
{foreign_destiny_id}-{vehicle_id}
```

El trigger `sp_tctb_insert` ejecuta `buscar_booking()` que compara esta llave contra todos los TtpnBookings del mismo día. El cuadre es transaccional (todo en la misma BD, mismo proceso = ~2ms). Separar en microservicios haría esto 150-400ms y rompería la atomicidad.

Cálculo de Nómina

```
Pago chofer = viajes_del_período * costo_por_viaje
           + incremento_por_nivel
           + incremento_por_cliente
           + incremento_por_servicio
           - deducciones
           - días_de_falta
```

Todas las funciones PG (`pago_chofer`, `incremento_por_nivel`, etc.) se ejecutan en el contexto de una `payroll_id` específica.

Haversine para Auto-sugerencia de Planta

```
// En app móvil, radio de 300 metros:
function calcularDistancia(lat1, lon1, lat2, lon2) {
  const R = 6371
  const dLat = (lat2 - lat1) * Math.PI / 180
  const dLon = (lon2 - lon1) * Math.PI / 180
  const a = Math.sin(dLat/2) ** 2 +
            Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI /
180) *
            Math.sin(dLon/2) ** 2
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a))
}
// Si distancia <= 0.3 km → auto-seleccionar esa planta
```

---

24. Reglas y Restricciones Globales

Datos

1. Todo registro operativo tiene `business_unit_id` — nunca se comparten datos entre BUs
2. Los choferes (`Employee.status = true`) son los únicos que pueden tener `VehicleAsignation` activa
3. Solo puede haber **una** `VehicleAsignation` activa por vehículo (fecha_hasta = nil)
4. Un `TravelCount` solo puede cuadrarse con **un** `TtpnBooking` y viceversa
5. Una `Payroll` abierta absorbe automáticamente nuevos `TravelCount` via `buscar_nomina()`
6. Las funciones PG (`buscar_booking`, `buscar_nomina`, etc.) son la fuente de verdad del cuadre

Seguridad

7. JWT de `User` y JWT de `Employee` son incompatibles — namespaces de rutas diferentes
8. Los tokens se revocan inmediatamente al logout regenerando el `jti`
9. El IMEI para binding de dispositivo está descartado (Android 10+ lo bloquea); se usa AndroidID como alternativa
10. Los endpoints de admin no son accesibles con token de chofer y viceversa

Versiones de App

11. Máximo 2 versiones activas simultáneas por dispositivo (móvil o web)
12. Solo puede haber una versión permanente (sin `fecha_fin`) activa por dispositivo
13. Para crear una nueva versión permanente, primero hay que ponerle `fecha_fin` futura a la actual

Frontend

14. El patrón estándar es: **Page orquestador + composables + componentes atómicos + AppTable + FilterPanel**
15. No hay validaciones de acceso hardcodeadas por `role_id` en el FE — todo se basa en el objeto `privileges` del JWT
16. Todos los errores de la API se muestran con `notifyApiError(error)` que extrae `error.response.data.errors`

Para más contexto sobre decisiones de arquitectura ver:

- ARQUITECTURA_MONOLITO_VS_MICROSERVICIOS.md
- INFRAESTRUCTURA_RAILWAY_NETLIFY_SUPABASE.md
- PROPUESTA_REGISTERED_APPS.md
- MIGRACION_PHP_A_RUBY.md